This lab implements a two-stage content retrieval system for video data. The workflow indexes a YouTube RAV4 review video by detecting car part instances using a YOLO segmentation model, then retrieves matching segments from the video based on query images from a curated dataset. The system demonstrates practical challenges in object detection, temporal alignment, and cross-domain matching.

The YOLO segmentation model used for indexing exhibits the following characteristics. Architecture: 139 layers, 2,693,369 parameters, 9.0 GLFOPS. For overal detection performance, the bounding box mAP50 was 0.665 and the istance mask mAP50 was 0.683. These metrics indicate moderate overall performance across all classes, but this aggregate figure masks significant variate in per-class accuracy. The model was trained on the carparts-seg dzataset with imbalance class frequencies, leading to highly heterogeneous performance across car part categories.

The sampling strategy extracts frames from a RAV4 review video (YouTube ID: YcvECxtXoxQ) at regular intervals of 5 seconds. Frame files in the frames/ directory are named with indices (e.g., frame_0000.jpg , frame_0001.jpg), and the detect.py script parses these indices to compute corresponding timestamps. For example, frame 0 represents 0 seconds, frame 1 represents 5 seconds, frame 10 represents 50 seconds, and so on.

The 5-second sampling intervale represents a practical tradeoff between competing objectives. It captures sufficient temporal diversity to observe the vehicle from multiple angles and under varying lighting conditions as the review progresses. This ensures that most car parts are visible at least once. And assuming a 30 fps video, processing frames every 5 seconds reduces the total fram count by a factor of ~150, making YOLO inference tractable on standard hardware.

The interval is coarse enough to prevent overlapping detections of stationary objects (which would unnecessarily inflate the index), yet fine enoughto distinguish between distinct vehicle orientations and scenes within the video.

The script reports: print(f"frames to process: {len(frame_files)}") . I got 600 raw frams from the ~45 minute YouTube review.

The detect.py script builds a searchable index of video content through the following steps:

1. Model Loading: A pre-trained YOLO segmentation model is loaded from runs/segment/train2/weights/best.pt. This model has been trained to recognize car part classes (hood, door, window, windshield, etc.) on the custom carparts-seg dataset.
2. Frame Iteration: For each JPG file in the frames/ directory (processed in sorted order), I extracted fram index from fileneame (e.g frame_0042.jpg has index=42), computed temporal coordinates: timestamp_sec = frame_index x 5, and executed YOLO inference on the frame image.
3. Detection Extraction: For each bounding box returned by YOLO, it recorded class_label(predicated car part category), confidence (model confidence score), normalized spatial coordinates (x_min, y_min, x_max, y_max), and metadata (video_id, frame_index, timestamp_sec)

All detection rows are accumulated into a list of dictionaries and serialized to detections.parquet. This creates a queryable table where each row corresponds to one detected object instance.

The retrieve.py script retrieves video segments matching query images through the following steps:

1. Dataset Loading: Loads detections from detections.parquet and a separate Hugging Face dataset of high-quality RAV4 exterior images (aegean-ai/rav4-exterior-images).
2. Query Image Processing: For each query image, the script executed YOLO predictions to identify visible car parts, collected all detected class labels into a set (e.g {"hood", "windshield", "headlight"}), filtered the video detections table to retrieve all rows where class-label matches any query label: matches = yt_dataset[yt_dataset["class_label"].isin(query_labels)], and produced candidate timestamps from the video.
3. Temporal Interval Grouping: It grouped matching timestamps into contiguous intervals, tolerating up to a 5-second gap between consecutive detections by initializing interva with first matched timestamp, extending interval while subsequent timestamps maintain

gap <= 5 seconds, and when the gap > 5 seconds is encountered, it closes the interval and starts a new one. The result is a list of time ranges (e.g [(0, 10), (25, 350), (120, 140)].

4. Output Generation: For each contiguous interval, I printed interval bounds (start and end timestamps in seconds) and count of detections within the interval. For example, [0 - 10] 15 detections indicates 15 car part instances matching the query were detected during the 0-10 second window.

The YOLO model exhibits sever class imbalance artifacts as shown by training data frequencies and per-class performance divergence below.

Class	Training Instances	mAP50
front_glass	214	0.971
front_bumper	208	0.960
hood	214	0.956
front_right_door	12	0.442
back_right_door	12	0.438
left_mirror	31	0.396

Well-represented classes (front_glass, hood) achieve ~96% mAP50, while rare classes (back_right_door: 12 instances) drop to 0.438 mAP450. That is a 2.4x performance gap. Many actual instances of rare parts in the video remain undetected (poor recall). False positives cluster on edge cases where the model has minimal training signal (poor precision). And the parquet file severely underrepresents rare classes, causing queries for “back_right_door” or “left_mirror” to have high false-negatives (incomplete index).

The matching mechanism treated all instance sof the same class as interchangeable. If a query image contains a “hood” and the video contains 50 frames with hoods all 50 are returned regardless of appearance similarity. There is no visual feature comparison like embedding-based similarity, and same class label is matched across arbitrary view angles, lighting conditions and contexts. So there is a high false positive rate when parts are present but visually dissimilar.

In conclusion, this lab demonstrates end-to-end object detection–based video retrieval. While functional, the system reveals key practical challenges: class imbalance in training data, the need for appearance-based refinement beyond class labels, temporal alignment subtleties, and domain shift effects. Future iterations should incorporate confidence thresholding, appearance similarity metrics, and adaptive temporal grouping.

References:

Ultralytics YOLO Documentation: https://docs.ultralytics.com/
Hugging Face Datasets: https://huggingface.co/datasets
COCO Evaluation Metrics (mAP): https://cocodataset.org